# Software Testing

Complete Study Notes — All 5 Units

B.Sc. (Information Technology) | Semester VI

★ **IMP:** Important topic — study thoroughly

**HIGH CHANCE:** Very likely to appear in exams — must know

## UNIT I: Basics of Software Testing — 12 Hours

### 1. Software Testing Fundamentals

**Definition:** Software Testing is the process of evaluating and verifying that a software system meets specified requirements and does not have unintended behavior. Goal: Find bugs before delivery to end users.

**Error → Defect → Failure Chain:**

- Error: A mistake made by the developer while writing code
- Defect / Bug: The result of an error found in the code
- Failure: When the software is unable to perform the required function

**7 Principles of Software Testing (HIGH CHANCE):**

- Testing shows presence of defects — not absence
- Exhaustive testing is impossible
- Early testing saves cost and time
- Defect clustering: 80% of bugs found in 20% of modules (Pareto principle)
- Pesticide Paradox: Same test cases stop finding new bugs over time
- Testing is context-dependent
- Absence of errors fallacy: Bug-free software may still fail to meet user needs

### 2. Verification vs Validation

| Aspect | Verification | Validation |
|---|---|---|
| Question | Are we building the product right? | Are we building the right product? |
| Type | Static Testing | Dynamic Testing |
| Execution | No code execution needed | Actual execution of software |

| | | |
|---|---|---|
| Done by | Developers / QA team | QA team / End users |
| Examples | Reviews, inspections, walkthroughs | Unit, integration, system testing |

## 3. SDLC vs STLC

| SDLC | STLC |
|---|---|
| Requirement Analysis | Requirement Analysis |
| System Design | Test Planning |
| Implementation/Coding | Test Case Design |
| Testing | Test Environment Setup |
| Deployment | Test Execution |
| Maintenance | Test Closure |

## ★ 4. Types of Testing

- Manual Testing: Human executes test steps one by one without automation
- Automation Testing: Tools (Selenium, etc.) automatically execute test cases
- Smoke Testing: Basic build verification test — go/no-go decision on a new build
- Sanity Testing: Quick, narrow check after a minor fix or change
- Regression Testing: Re-testing after code changes to ensure nothing is broken
- Ad-hoc Testing: Random, unplanned testing without any documentation

**Smoke vs Sanity:** Smoke = broad & shallow check on entire build. Sanity = narrow & deep check on specific function.

# UNIT II: Testing Techniques and Levels — 12 Hours

## 1. Black Box Testing

**Definition:** Testing based on functional requirements without knowledge of internal code. Tester sees only inputs and outputs.

### Equivalence Class Partitioning (ECP):

Divide input data into valid and invalid partitions. Test one representative value from each class.

- Example: Age field accepts 1–100
- Valid partition: 1–100 → Test: 50
- Invalid partitions: <1 and >100 → Test: 0, 101

### Boundary Value Analysis (BVA):

Test values at the boundaries/edges of input partitions.

- For Age 1–100: Test values 0, 1, 2, 99, 100, 101

- BVA covers the boundary itself plus one above and one below

**Decision Table Testing:**

- Used for business rules with multiple conditions and corresponding actions
- Each column = one rule; rows = conditions & actions

**State Transition Testing:**

- Tests behavior as the system moves between states based on inputs/events

## 2. White Box Testing

**Definition:** Tests internal logic, code structure. Tester has full access to the source code. Also called Glass Box or Structural Testing.

| Coverage Type | Description |
|---|---|
| Statement Coverage | Every line/statement of code executed at least once |
| Branch Coverage | Every true/false branch of decision points tested |
| Path Coverage | Every possible path through the code executed |
| Condition Coverage | Each boolean sub-condition tested true AND false |

## ★ 3. Testing Levels

| Level | Description | Done by |
|---|---|---|
| Unit Testing | Test individual components/functions in isolation | Developers |
| Integration Testing | Test interaction/interfaces between modules | Developers/QA |
| System Testing | Test the complete, integrated system | QA Team |
| Acceptance Testing (UAT) | Validate system meets business requirements | End users/Client |

**Integration Testing Approaches:**

- Big Bang: All modules combined and tested at once
- Top-Down: Testing from top module downward using stubs
- Bottom-Up: Testing from bottom module upward using drivers
- Sandwich/Hybrid: Combination of top-down and bottom-up

## ★ 4. Non-Functional Testing Types

- Performance Testing: Load, Stress, Spike, Endurance tests
- Security Testing: Checks for vulnerabilities, unauthorized access, SQL injection
- Usability Testing: Evaluates user-friendliness and UX of the application
- Compatibility Testing: Tests across different browsers, devices, OS versions

# UNIT III: Software Testing Management

12 Hours

## 1. Test Case Design

**Definition:** A Test Case is a document that specifies inputs, execution steps, expected results, and actual results for a specific scenario.

| Field | Description |
|---|---|
| Test Case ID | Unique identifier for the test case |
| Test Title/Summary | Brief description of what is being tested |
| Preconditions | State that must exist before executing the test |
| Test Steps | Step-by-step actions to perform |
| Expected Result | What should happen if software works correctly |
| Actual Result | What actually happened during execution |
| Status | Pass / Fail |
| Priority | High / Medium / Low |

**Test Case vs Test Scenario:** Scenario = WHAT to test (high level). Test Case = HOW to test (detailed step-by-step).

## 2. Test Plan vs Test Strategy

| Test Plan | Test Strategy |
|---|---|
| Project-level document | Organization-level document |
| Created by Test Lead/Manager | Created by QA Manager/Architect |
| Includes scope, resources, schedule | Includes approach, tools, types of tests |
| Specific to one project | Applies to multiple projects |

**Test Plan Contents:**

- Scope and objectives
- Test approach and types of testing
- Resources and roles
- Entry and Exit criteria
- Schedule and milestones
- Risks and mitigation
- Deliverables

## 3. Defect Life Cycle

**Lifecycle:** New → Assigned → Open → Fixed → Retest → Closed (or Reopen if defect still exists after fix)

| Concept | Definition | Example |
|---|---|---|
| Severity | Impact of defect on the system | Critical: App crashes; Minor: Typo |
| Priority | Urgency to fix the defect | High: Must fix now; Low: Fix later |

**Key Insight:** High Severity + Low Priority: Crash in rarely-used feature. Low Severity + High Priority: Company logo broken on homepage.

## ★ 4. Defect Tracking Tools

- JIRA: Most popular; agile project management + defect tracking
- Bugzilla: Open-source web-based defect tracking by Mozilla
- Mantis: Web-based open source bug tracking system
- Redmine: Project management + issue tracking combined

## ★ 5. Test Metrics

| Metric | Formula |
|---|---|
| Defect Density | Total Defects / Size of Software |
| Test Coverage | (Tested Items / Total Items) × 100 |
| Defect Removal Efficiency | (Defects found before release / Total defects) × 100 |
| Pass Rate | (Passed Test Cases / Total Test Cases) × 100 |

# UNIT IV: Test Automation and Frameworks — 12 Hours

## 1. Introduction to Automation Testing

**Definition:** Using tools and scripts to automatically execute test cases, compare results, and report outcomes. Ideal for repetitive and regression tests.

| When to Automate | When NOT to Automate |
|---|---|
| Repetitive/regression tests | One-time or ad-hoc tests |
| Large data sets (data-driven) | Exploratory or usability testing |
| Performance/load testing | Frequently changing UI |
| Cross-browser tests | Tests requiring human judgment |

## 2. Selenium WebDriver

**Definition:** Open-source web browser automation tool. Supports multiple languages (Java, Python, C#) and browsers (Chrome, Firefox, Edge).

**Architecture:**

Selenium Client Library → WebDriver Protocol (W3C/JSON Wire) → Browser Driver → Browser

**Key Commands:**

| Command | Purpose |
|---|---|
| driver.get(url) | Open a URL in the browser |
| driver.findElement(By.id()) | Locate an element by ID |
| element.sendKeys("text") | Type text into input field |
| element.click() | Click on an element |
| element.getText() | Retrieve visible text of element |
| driver.quit() | Close browser and end session |

**Locators (in order of preference):**

- id — Fastest, most reliable if available
- name — Next preference
- cssSelector — Fast and flexible
- xpath — Most powerful, handles complex scenarios
- className, tagName, linkText, partialLinkText

## 3. Page Object Model (POM)

**Definition:** A design pattern that creates a class for each web page, containing its UI element locators and related actions. Separates test logic from UI locators.

- Each web page = One separate class (e.g., LoginPage.java)
- Element locators defined in page class, not in test class
- Test class calls methods from page class

**Advantages:** Code reusability, easy maintenance when UI changes, cleaner and readable tests, reduced duplication.

**Page Factory:**

- Extension of POM using @FindBy annotations
- Elements initialized lazily using PageFactory.initElements(driver, this)
- Cleaner syntax compared to traditional POM

## ★ 4. TestNG Framework

**Definition:** Testing framework for Java inspired by JUnit. Supports grouping, parallel execution, annotations, and data-driven testing.

| Annotation | Purpose |
|---|---|
| @Test | Marks a method as a test case |
| @BeforeMethod | Runs before each test method |
| @AfterMethod | Runs after each test method |
| @BeforeClass | Runs once before first test in class |
| @AfterClass | Runs once after last test in class |
| @DataProvider | Supplies test data for data-driven tests |
| @Parameters | Pass parameters from testng.xml file |

## ★ 5. JUnit Basics

- @Test — Mark method as test
- @Before / @After — Setup and teardown for each test
- assertEquals(expected, actual) — Check equality
- assertTrue(condition) / assertFalse(condition)

| Feature | JUnit | TestNG |
|---|---|---|
| Parallel execution | Limited | Built-in support |
| Data-driven testing | Via @Parameterized | Via @DataProvider (easier) |
| Grouping tests | Limited | Full support |
| Annotations | Simpler set | More comprehensive |

# UNIT V: Agile Testing and Industry Practices — 12 Hours

## 1. Agile Testing

**Definition:** Testing in Agile is continuous, iterative, and collaborative. Testers work alongside developers within each sprint, providing immediate feedback.

**Key Agile Testing Principles:**

- Early testing — start in sprint planning, not at end
- Continuous feedback — immediate defect communication to developers
- Test-first approach — write tests before code (TDD)
- Whole team owns quality — not just the QA team

**Agile Testing Quadrants:**

| Quadrant | Type | Tools |
|---|---|---|
| Q1 (Technology-facing, supports team) | Unit & Component tests | JUnit, NUnit |
| Q2 (Business-facing, supports team) | Functional & Story tests | Cucumber, FitNesse |
| Q3 (Business-facing, critiques product) | Exploratory, UAT, Usability | Manual testing |
| Q4 (Technology-facing, critiques product) | Performance, Security, Load | JMeter, OWASP ZAP |

## 2. BDD with Cucumber

**BDD Definition:** Behavior Driven Development — Tests written in plain English (Gherkin syntax) that all stakeholders (developers, testers, business) can understand.

### Gherkin Syntax (Very Likely Exam Question):

| Keyword | Purpose | Example |
|---|---|---|
| Feature | Describes the feature being tested | Feature: User Login |
| Scenario | Describes one specific test scenario | Scenario: Valid login |
| Given | Pre-condition / initial state | Given user is on login page |
| When | Action performed by user/system | When user enters valid credentials |
| Then | Expected outcome/result | Then user is redirected to dashboard |
| And / But | Add more steps to Given/When/Then | And user clicks submit button |

## ★ 3. CI/CD in Testing

**CI:** Continuous Integration — Code is automatically built and tested on every commit to detect issues early.

**CD:** Continuous Delivery/Deployment — After passing tests, code is automatically deployed to staging or production.

### CI/CD Pipeline with Jenkins:

Developer commits code → Jenkins triggers build → Runs unit tests → Runs Selenium tests → Generates report → Deploys if all pass

## ★ 4. Performance Testing with JMeter

**Apache JMeter:** Open-source Java tool for load testing and measuring performance of web applications.

| Test Type | Description |
|---|---|
| Load Testing | Test under normal expected user load |
| Stress Testing | Test beyond maximum capacity until failure |
| Spike Testing | Sudden sharp increase in user load |
| Endurance/Soak Testing | Sustained load over extended period |

**JMeter Key Components:**

- Thread Group: Defines number of users, ramp-up time, loop count
- Samplers: HTTP Request samplers send actual requests
- Listeners: Collect and display test results (View Results Tree, Summary Report)
- Assertions: Validate response (Response Assertion, Duration Assertion)
- Timers: Add delays between requests to simulate real user behavior

### ★ 5. Shift-Left Testing & DevOps

- Shift-Left Testing: Test as early as possible in the SDLC to reduce cost
- DevOps Pipeline: Plan → Code → Build → Test → Release → Deploy → Operate → Monitor

**Best Practices:**

- Automate wherever possible — unit, integration, regression
- Test early and often in every sprint
- Maintain good test coverage (aim for 80%+)
- Prioritize testing of high-risk areas
- Keep test scripts version-controlled in Git

## Quick Revision — Most Important Topics

| Unit | Must-Know Topics for Exams |
|---|---|
| Unit I | 7 Principles of Testing, Verification vs Validation (table), SDLC vs STLC phases |
| Unit II | BVA & ECP with numerical examples, Black Box vs White Box differences, Testing Levels |
| Unit III | Defect Life Cycle diagram, Test Case format with all fields, Severity vs Priority with examples |
| Unit IV | Selenium locators & WebDriver architecture, POM concept, TestNG annotations |
| Unit V | Gherkin Given-When-Then with example, CI/CD pipeline stages, Agile testing principles |

**Pro Tip:** For Unit II — always solve a numerical example using ECP and BVA (e.g., 'Design test cases for a login form with age field 18-60'). These carry high marks. For Unit V — write a complete Gherkin scenario from scratch.